Experiment 9

AIM: Use pytest to test python application.

1. Objective

- Understand unit testing concepts
- Write test cases using PyTest
- Execute and analyze test results
- Handle edge cases and exceptions
- Use parameterized testing

2. Prerequisites

- Python
- Pytest

3. Installation

Install pytest using pip:

```
pip install pytest
```

Verify it :

```
pytest –version
```

4. Create a Python module with basic mathematical operations and write PyTest test cases to validate:

- Correct outputs
- Edge cases
- Exception handling

Step 1: Create Project PythonDemo and add file pythondemo.py

```
def add(a, b):
 return a + b
```

```
def subtract(a, b):
 return a - b
def multiply(a, b):
 return a * b
def divide(a, b):
 if b == 0:
 raise ValueError("Cannot divide by zero")
 return a / b
def is_even(num):
 return num % 2 == 0
```

Step 2: Create a test file test_pythondemo.py

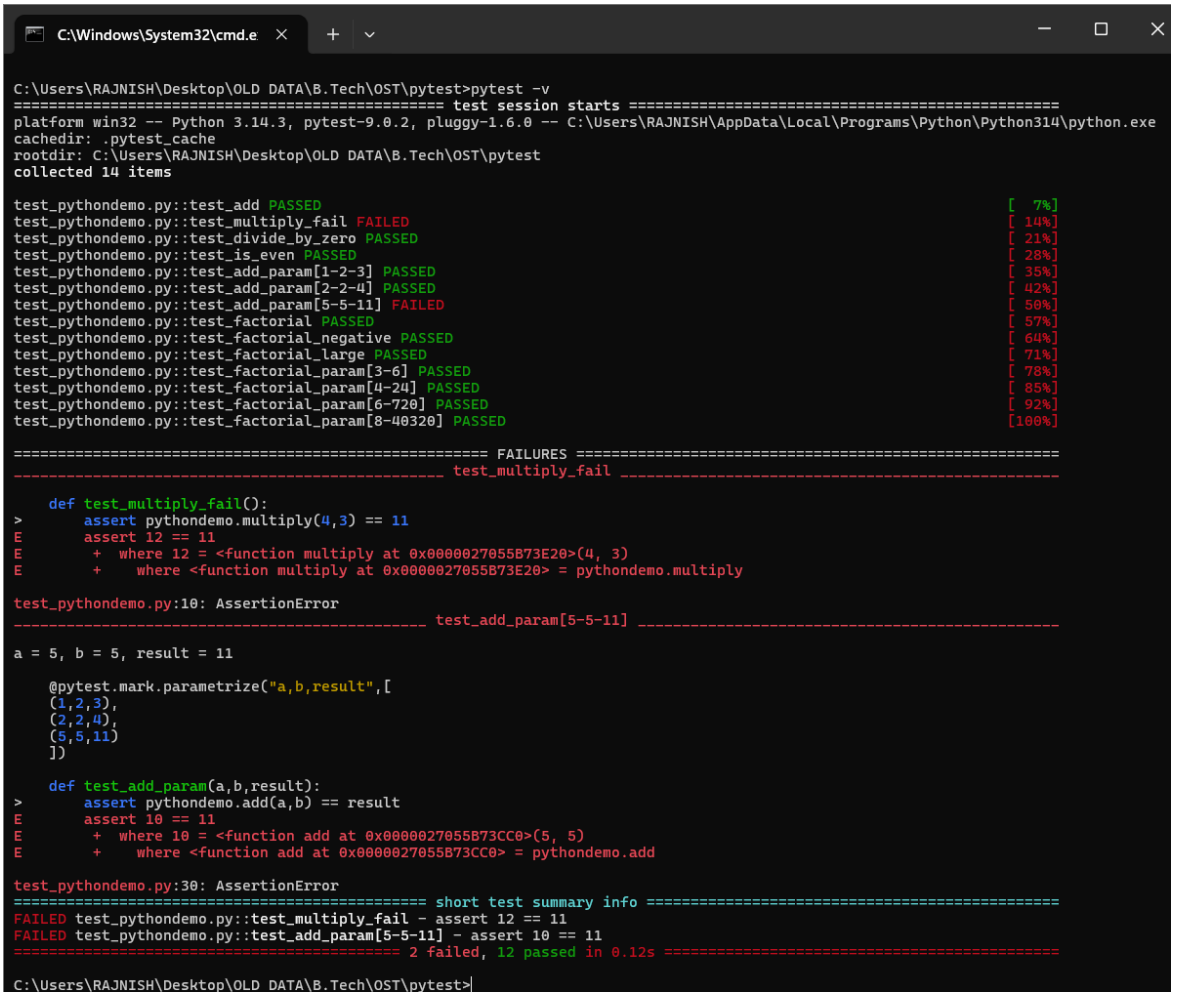
```
import pytest
import pythondemo
# Passing test
def test_add():
 assert pythondemo.add(2, 3) == 5
# Failing test
def test_multiply_fail():
 assert pythondemo.multiply(4, 3) == 11
# Exception test
def test_divide_by_zero():
 with pytest.raises(ValueError):
 pythondemo.divide(10, 0)
# Logical test
def test_is_even():
 assert pythondemo.is_even(4) is True
```

```
 assert pythondemo.is_even(5) is False

# Parametrized test

@pytest.mark.parametrize("a,b,result", [

 (1,2,3),

 (2,2,4),

 (5,5,11) # failing case

])

def test_add_param(a,b,result):

 assert pythondemo.add(a,b) == result
```

5. Execution

Open terminal in that folder

pytest -v



Task 1 and 2:

1. PythonDemp.py

```
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b


def is_even(num):
    return num % 2 == 0



# Task 1 - factorial function
def factorial(n):
    if n < 0:
        raise ValueError("Negative numbers not allowed")
```

```
    result = 1
    for i in range(1, n + 1):
        result *= i

    return result
```

2. Test_PythonDemo.py

```
import pytest
import PythonDemo


# Passing test
def test_add():
    assert PythonDemo.add(2, 3) == 5


# Failing test
def test_multiply_fail():
    assert PythonDemo.multiply(4, 3) == 11


# Exception test
def test_divide_by_zero():
    with pytest.raises(ValueError):
        PythonDemo.divide(10, 0)
```

```
# Logical test
def test_is_even():
    assert PythonDemo.is_even(4) is True
    assert PythonDemo.is_even(5) is False



# Parameterized test
@pytest.mark.parametrize("a,b,result", [
    (1,2,3),
    (2,2,4),
    (5,5,11)   # failing case
])
def test_add_param(a,b,result):
    assert PythonDemo.add(a,b) == result



# Task 1 – factorial tests
def test_factorial():
    assert PythonDemo.factorial(5) == 120
    assert PythonDemo.factorial(0) == 1



# Task 2 – negative number test
def test_factorial_negative():
    with pytest.raises(ValueError):
```

```
        PythonDemo.factorial(-5)
```

```
# Task 2 – large input test

def test_factorial_large():

    assert PythonDemo.factorial(10) == 3628800
```

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\RAJNISH\Desktop\OLD DATA\B.Tech\OST\pytest>pytest -v
================================================= test session starts =================================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\RAJNISH\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\RAJNISH\Desktop\OLD DATA\B.Tech\OST\pytest
collected 10 items

test_pythondemo.py::test_add PASSED                                                                                  [ 10%]
test_pythondemo.py::test_multiply_fail FAILED                                                                        [ 20%]
test_pythondemo.py::test_divide_by_zero PASSED                                                                       [ 30%]
test_pythondemo.py::test_is_even PASSED                                                                              [ 40%]
test_pythondemo.py::test_add_param[1-2-3] PASSED                                                                     [ 50%]
test_pythondemo.py::test_add_param[2-2-4] PASSED                                                                     [ 60%]
test_pythondemo.py::test_add_param[5-5-11] FAILED                                                                    [ 70%]
test_pythondemo.py::test_factorial PASSED                                                                            [ 80%]
test_pythondemo.py::test_factorial_negative PASSED                                                                   [ 90%]
test_pythondemo.py::test_factorial_large PASSED                                                                      [100%]

======================================================= FAILURES ========================================================
_____________________________________________________ test_multiply_fail _____________________________________________________

    def test_multiply_fail():
>       assert pythondemo.multiply(4, 3) == 11
E       assert 12 == 11
E        +  where 12 = <function multiply at 0x00000231CA33F3D0>(4, 3)
E        +    where <function multiply at 0x00000231CA33F3D0> = pythondemo.multiply

test_pythondemo.py:12: AssertionError
_____________________________________________________ test_add_param[5-5-11] _____________________________________________________

a = 5, b = 5, result = 11

    @pytest.mark.parametrize("a,b,result", [
    (1,2,3),
    (2,2,4),
    (5,5,11)   # failing case
    ])

    def test_add_param(a,b,result):
>       assert pythondemo.add(a,b) == result
E       assert 10 == 11
E        +  where 10 = <function add at 0x00000231CA33F270>(5, 5)
E        +    where <function add at 0x00000231CA33F270> = pythondemo.add

test_pythondemo.py:35: AssertionError
=============================================== short test summary info ================================================
FAILED test_pythondemo.py::test_multiply_fail - assert 12 == 11
FAILED test_pythondemo.py::test_add_param[5-5-11] - assert 10 == 11
============================================== 2 failed, 8 passed in 0.09s ================================================
```

Command:

C:\Users\RAJNISH\Desktop\OLD DATA\B.Tech\OST\pytest>pytest -v
========== test session starts
==========
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 --
C:\Users\RAJNISH\AppData\Local\Programs\Python\Python314\python.exe

```
cachedir: .pytest_cache
rootdir: C:\Users\RAJNISH\Desktop\OLD DATA\B.Tech\OST\pytest
collected 10 items

test_pythondemo.py::test_add PASSED                                    [ 10%]
test_pythondemo.py::test_multiply_fail FAILED                          [ 20%]
test_pythondemo.py::test_divide_by_zero PASSED                         [ 30%]
test_pythondemo.py::test_is_even PASSED                                [ 40%]
test_pythondemo.py::test_add_param[1-2-3] PASSED                       [ 50%]
test_pythondemo.py::test_add_param[2-2-4] PASSED                       [ 60%]
test_pythondemo.py::test_add_param[5-5-11] FAILED                      [ 70%]
test_pythondemo.py::test_factorial PASSED                              [ 80%]
test_pythondemo.py::test_factorial_negative PASSED                     [ 90%]
test_pythondemo.py::test_factorial_large PASSED                        [100%]

=================================================== FAILURES ===================================================
_____________________________________________ test_multiply_fail _____________________________________________

    def test_multiply_fail():
>       assert pythondemo.multiply(4, 3) == 11
E       assert 12 == 11
E        +  where 12 = <function multiply at 0x00000231CA33F3D0>(4, 3)
E        +    where <function multiply at 0x00000231CA33F3D0> = pythondemo.multiply

test_pythondemo.py:12: AssertionError
_____________________________________________ test_add_param[5-5-11] _____________________________________________

a = 5, b = 5, result = 11

    @pytest.mark.parametrize("a,b,result", [
    (1,2,3),
    (2,2,4),
    (5,5,11)   # failing case
    ])

    def test_add_param(a,b,result):
>       assert pythondemo.add(a,b) == result
E       assert 10 == 11
E        +  where 10 = <function add at 0x00000231CA33F270>(5, 5)
E        +    where <function add at 0x00000231CA33F270> = pythondemo.add

test_pythondemo.py:35: AssertionError
=============================================== short test summary info ================================================
FAILED test_pythondemo.py::test_multiply_fail - assert 12 == 11
FAILED test_pythondemo.py::test_add_param[5-5-11] - assert 10 == 11
============================================= 2 failed, 8 passed in 0.09s ==============================================
```